Национальный исследовательский университет
Высшая школа экономики
Московский институт электроники и математики

Департамент прикладной математики
Кафедра компьютерной безопасности

Лабораторная работа №1
по Параллельным вычислениям

Освоение векторизации

Вариант 2

Выполнил:
Ишкинин Д.С.

Проверил:
Байдин Г.С.

Москва 2025

Содержание

1 Постановка задачи **2**

2 Описание используемой вычислительной системы и компилятора **2**

2.1 Вычислительная система 2

2.2 Компилятор 2

3 Ссылка на репозиторий **3**

4 Результаты выполнения задания **3**

4.1 Исходная программа, без оптимизации 3

4.2 Исходная программа, оптимизированная только с помощью ключей компилятора 4

4.3 Программа, оптимизированная полуавтоматически . . 5

5 Таблица результатов **6**

6 Листинг на ассемблере фрагмента кода **6**

7 Вывод по результатам выполнения задания **13**

1 Постановка задачи

Реализовать транспонирование плотных матриц больших размерностей.

Оптимизировать программу автоматически (ключами компилятора) и полуавтоматически (директивами компилятора и ключами компилятора).

Сравнить время работы следующих вариантов программы:

- исходная программа, без оптимизации
- исходная программа, оптимизированная только с помощью ключей компилятора
- программа, оптимизированная полуавтоматически

2 Описание используемой вычислительной системы и компилятора

2.1 Вычислительная система

- Процессор: Intel Core i7
- Количество ядер процессора: 10
- Оперативная память: 16 ГБ DDR5
- Видеокарта: NVIDIA GeForce RTX 3050 Ti
- Тип системы: 64-разрядная операционная система, процессор x64
- Операционная система: Windows 10 (22H2)

2.2 Компилятор

GCC 15.2.0

3 Ссылка на репозиторий

Ссылка на GitHub репозиторий. В репозитории лежат следующие файлы:

- lab.cpp - базовая программа без оптимизаций
- lab_optim.cpp - программа с полуавтоматической оптимизацией
- lab_optim_asm.s - ассемблерный код оптимизированной программы

4 Результаты выполнения задания

4.1 Исходная программа, без оптимизации

```
PS C:\Users\Daniel\VSCodeProjects\lab1> g++ lab.cpp -o lab.exe
PS C:\Users\Daniel\VSCodeProjects\lab1> ./lab.exe
Iteration 1 time: 1239 ms
Iteration 2 time: 1299 ms
Iteration 3 time: 1205 ms
Iteration 4 time: 1232 ms
Iteration 5 time: 1262 ms
Iteration 6 time: 1379 ms
Iteration 7 time: 1462 ms
Iteration 8 time: 1430 ms
Iteration 9 time: 1252 ms
Iteration 10 time: 1293 ms

Average transpose time: 1305.3 ms
```

4.2 Исходная программа, оптимизированная только с помощью ключей компилятора

```
PS C:\Users\Daniel\VSCodeProjects\lab1> g++ lab.cpp -o lab_keys.exe -march=native -O3
PS C:\Users\Daniel\VSCodeProjects\lab1> ./lab_keys.exe
Iteration 1 time: 586 ms
Iteration 2 time: 573 ms
Iteration 3 time: 593 ms
Iteration 4 time: 576 ms
Iteration 5 time: 590 ms
Iteration 6 time: 573 ms
Iteration 7 time: 588 ms
Iteration 8 time: 577 ms
Iteration 9 time: 588 ms
Iteration 10 time: 572 ms

Average transpose time: 581.6 ms
```

Использованы следующие ключи компилятора:

- Ключ -O3, задающий максимальный уровень оптимизации. Включает все оптимизации уровней O1 и O2, а также выполняет агрессивные оптимизации.
- Ключ -march, задающий целевую архитектуру процессора. Указание значения native позволяет автоматически определить архитектуру текущего процессора при компиляции.

4.3 Программа, оптимизированная полуавтоматически

```
PS C:\Users\Daniel\VSCodeProjects> g++ -fopenmp -O3 -march=native lab_optim.cpp -o lab_optim.exe
PS C:\Users\Daniel\VSCodeProjects> ./lab_optim.exe
Iteration 1 time: 216 ms
Iteration 2 time: 213 ms
Iteration 3 time: 231 ms
Iteration 4 time: 234 ms
Iteration 5 time: 217 ms
Iteration 6 time: 213 ms
Iteration 7 time: 239 ms
Iteration 8 time: 202 ms
Iteration 9 time: 199 ms
Iteration 10 time: 209 ms

Average transpose time: 217.3 ms
```

В коде программы используется директива векторизации цикла на основе стандарта OpenMP. Также используется флаг компиляции -fopenmp, необходимый для включения поддержки OpenMP в компиляторе.

Директива ***#pragma omp parallel for collapse(2)*** состоит из нескольких частей:

- #pragma omp - указывает компилятору, что это директива OpenMP
- parallel - создает команду параллельных потоков для выполнения кода
- for - распределяет итерации следующего цикла между созданными потоками
- collapse(2) - объединяет два вложенных цикла в единое пространство итераций для равномерного распределения работы между потоками

5 Таблица результатов

Оптимизация	Время выполнения, мс	Ускорение программы с оптимизацией (относительно исходной), раз
Нет	1305.3	1.0
Ключи компилятора	581.6	2.2
Полуавтоматическая оптимизация	217.3	6.0

6 Листинг на ассемблере фрагмента кода

```
        .def _Z9transposeRKSt6vectorIS_IiSaIiEESaIS1_EE._omp_fn.0;    .scl    3;      .type   32;     .endef
        .seh_proc _Z9transposeRKSt6vectorIS_IiSaIiEESaIS1_EE._omp_fn.0
_Z9transposeRKSt6vectorIS_IiSaIiEESaIS1_EE._omp_fn.0:
.LFB3851:
        push    r14      #
        .seh_pushreg    r14
        push    r12      #
        .seh_pushreg    r12
        push    rbp      #
        .seh_pushreg    rbp
        push    rdi      #
        .seh_pushreg    rdi
        push    rsi      #
        .seh_pushreg    rsi
        push    rbx      #
        .seh_pushreg    rbx
        sub     rsp, 40  #,
        .seh_stackalloc 40
        .seh_endprologue
 # lab_optim.cpp:25:     #pragma omp parallel for collapse (2)
```

```
        mov     ebx, DWORD PTR 16[rcx]   # rows, *.omp_data_i_9(D).rows
        test    ebx, ebx         # rows
        mov     rbp, rcx         # .omp_data_i, .omp_data_i
        jle     .L11     #,
        mov     esi, DWORD PTR 20[rcx]   # cols, *.omp_data_i_9(D).cols
        test    esi, esi         # cols
        jle     .L11     #,
        call    omp_get_num_threads      #
        mov     edi, eax         # _15, _15
        call    omp_get_thread_num       #
        xor     edx, edx         # tmp160
        mov     ecx, eax         # _18,
        mov     eax, ebx         # rows, rows
        imul    eax, esi         # rows, cols
        div     edi      # _15
        cmp     ecx, edx         # _18, tmp160
        lea     r8d, 1[rax]      # tmp199,
        cmovnb  r8d, eax         # tmp199,, q.19_1, tmp159
        mov     eax, 0   # tmp201,
        cmovb   edx, eax         # tmp160,, tt.20_2, tmp201
        imul    ecx, r8d         # _22, q.19_1
        add     ecx, edx         # _23, tt.20_2
        lea     r9d, [r8+rcx]    # _24,
        cmp     ecx, r9d         # _23, _24
        jnb     .L11     #,
        mov     eax, ecx         # i, _23
        xor     edx, edx         # tmp163
# C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32/15.2.0/include/c++/bits/stl_vector.h:1283:     return *(this->_M_impl._M_start + __n);
        mov     r11, QWORD PTR 0[rbp]    # *.omp_data_i_9(D).matrix, *.omp_data_i_9(D).matrix
# C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32/15.2.0/include/c++/bits/stl_vector.h:1264:     return *(this->_M_impl._M_start + __n);
        mov     rbx, QWORD PTR 8[rbp]    # *.omp_data_i_9(D).transposed, *.omp_data_i_9(D).transposed
        div     esi      # cols
```

```
# C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32/15.2.0/include/c++/bits/stl_vector.h:1283:     return *(this->_M_impl._M_start + __n);
        mov     rdi, QWORD PTR [r11]     # _60, *_59.D.72811._M_impl.D.72114._M_start
# C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32/15.2.0/include/c++/bits/stl_vector.h:1264:     return *(this->_M_impl._M_start + __n);
        mov     rbx, QWORD PTR [rbx]     # _69, *_68.D.72811._M_impl.D.72114._M_start
        add     ecx, 1   # .iter.14_76,
        cmp     ecx, r9d         # .iter.14_76, _24
# lab_optim.cpp:28:             transposed[j][i] = matrix[i][j];
        movsx   r10, eax         # _58, i
# lab_optim.cpp:28:             transposed[j][i] = matrix[i][j];
        mov     r12d, edx        # _63, tmp163
# C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32/15.2.0/include/c++/bits/stl_vector.h:1283:     return *(this->_M_impl._M_start + __n);
        lea     r11, [r10+r10*2]         # tmp168,
# C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32/15.2.0/include/c++/bits/stl_vector.h:1264:     return *(this->_M_impl._M_start + __n);
        lea     rbp, [r12+r12*2]         # tmp175,
# C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32/15.2.0/include/c++/bits/stl_vector.h:1283:     return *(this->_M_impl._M_start + __n);
        mov     r11, QWORD PTR [rdi+r11*8]       # prephitmp_110, MEM[(const struct vector *)_62].D.73852._M_impl.D.73191._M_start
# lab_optim.cpp:28:             transposed[j][i] = matrix[i][j];
        mov     rbp, QWORD PTR [rbx+rbp*8]       # MEM[(struct vector *)_71].D.73852._M_impl.D.73191._M_start, MEM[(struct vector *)_71].D.73852._M_impl.D.73191._M_start
# lab_optim.cpp:28:             transposed[j][i] = matrix[i][j];
        mov     r14d, DWORD PTR [r11+r12*4]      # _67, *_66
```

```
# lab_optim.cpp:28:                transposed[j][i] = matrix[i][j];
        mov     DWORD PTR 0[rbp+r10*4], r14d     # *_74, _67
        jnb     .L11     #,
        lea     ebp, -1[r8]      # _54,
        xor     r8d, r8d         # ivtmp.192
        .p2align 6
        .p2align 4,,10
        .p2align 3
.L9:
        add     edx, 1   # j,
        cmp     esi, edx         # cols, j
        jle     .L7      #,
# lab_optim.cpp:28:                transposed[j][i] = matrix[i][j];
        movsx   rcx, edx         # _102, j
        add     r8d, 1   # ivtmp.192,
# C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32/15.2.0/include/c++/bits/stl_vector.h:1264:     return *(this->_M_impl._M_start + __n);
        lea     r9, [rcx+rcx*2]  # tmp183,
# lab_optim.cpp:28:                transposed[j][i] = matrix[i][j];
        mov     r10d, DWORD PTR [r11+rcx*4]      # _90, *_98
        cmp     r8d, ebp         # ivtmp.192, _54
# lab_optim.cpp:28:                transposed[j][i] = matrix[i][j];
        movsx   rcx, eax         # _97, i
# lab_optim.cpp:28:                transposed[j][i] = matrix[i][j];
        mov     r9, QWORD PTR [rbx+r9*8]         # MEM[(struct vector *)_100].D.73852._M_impl.D.73191._M_start, MEM[(struct vector *)_100].D.73852._M_impl.D.73191._M_start
        mov     DWORD PTR [r9+rcx*4], r10d       # *_87, _90
        jne     .L9      #,
.L11:
# lab_optim.cpp:25:     #pragma omp parallel for collapse(2)
```

```
        add     rsp, 40  #,
        pop     rbx      #
        pop     rsi      #
        pop     rdi      #
        pop     rbp      #
        pop     r12      #
        pop     r14      #
        ret
```

```
        vpxor   xmm0, xmm0, xmm0         # tmp159
 # C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32
    /15.2.0/include/c++/bits/stl_vector.h:387:      return
     __n != 0 ? _Tr::allocate(_M_impl, __n) : pointer();
        test    rsi, rsi         # ivtmp.301
 # C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32
    /15.2.0/include/c++/bits/stl_vector.h:106:      :
    _M_start(), _M_finish(), _M_end_of_storage()
        vmovdqu XMMWORD PTR [r12], xmm0  # MEM <vector(2)
           long long unsigned int> [(struct vector * *)
           transposed_9(D)], tmp159
 # C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32
    /15.2.0/include/c++/bits/stl_vector.h:387:      return
     __n != 0 ? _Tr::allocate(_M_impl, __n) : pointer();
        je      .L103    #,
 # C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32
    /15.2.0/include/c++/bits/new_allocator.h:151:   return
     static_cast<_Tp*>(_GLIBCXX_OPERATOR_NEW(__n * sizeof(
    _Tp)));
        lea     rax, [rsi+rsi*2]         # _77,
        lea     rbx, 0[0+rax*8]  # tmp163,
        mov     rcx, rbx         #, tmp163
.LEHB6:
        call    _Znwy    #
.LEHE6:
 # C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32
    /15.2.0/include/c++/bits/stl_vector.h:405:      this->
    _M_impl._M_start = this->_M_allocate(__n);
        vmovq   xmm4, rax        # _78, _78
 # C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32
    /15.2.0/include/c++/bits/stl_vector.h:407:      this->
    _M_impl._M_end_of_storage = this->_M_impl._M_start +
    __n;
```

```
        mov     rdx, rax          # _78, _78
# C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32/15.2.0/include/c++/bits/new_allocator.h:151:   return static_cast<_Tp*>(_GLIBCXX_OPERATOR_NEW(__n * sizeof(_Tp)));
        mov     QWORD PTR 56[rsp], rax   # %sfp, _78
        sub     rdi, rbp          # _38, _61
# C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32/15.2.0/include/c++/bits/stl_vector.h:407:      this->_M_impl._M_end_of_storage = this->_M_impl._M_start + __n;
        lea     rax, [rax+rbx]   # __first,
# C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32/15.2.0/include/c++/bits/stl_vector.h:405:      this->_M_impl._M_start = this->_M_allocate(__n);
        vpunpcklqdq     xmm0, xmm4, xmm4         # _98, _78
# C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32/15.2.0/include/c++/bits/stl_vector.h:106:      :_M_start(), _M_finish(), _M_end_of_storage()
        vpxor   xmm6, xmm6, xmm6         # tmp167
# C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32/15.2.0/include/c++/bits/stl_vector.h:1119:   ptrdiff_t __dif = this->_M_impl._M_finish - this->_M_impl._M_start;
        mov     rbx, rdx          # __first, _78
# C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32/15.2.0/include/c++/bits/stl_vector.h:407:      this->_M_impl._M_end_of_storage = this->_M_impl._M_start + __n;
        mov     QWORD PTR 16[r12], rax   # MEM[(struct _Vector_base *)transposed_9(D)]._M_impl.D.72114._M_end_of_storage, __first
# C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32/15.2.0/include/c++/bits/stl_vector.h:405:      this->_M_impl._M_start = this->_M_allocate(__n);
        vmovdqu XMMWORD PTR [r12], xmm0  # MEM <vector(2) long long unsigned int> [(struct vector * *)transposed_9(D)], _98
        je      .L104    #,
        .p2align 4,,10
        .p2align 3
```

```
.L76:
 # C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32
    /15.2.0/include/c++/bits/stl_vector.h:106:      :
    _M_start(), _M_finish(), _M_end_of_storage()
        mov     QWORD PTR 16[rbx], 0     # MEM[(int * *)
           __first_49 + 16B],
 # C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32
    /15.2.0/include/c++/bits/new_allocator.h:151:   return
     static_cast<_Tp*>(_GLIBCXX_OPERATOR_NEW(__n * sizeof(
    _Tp)));
        mov     rcx, rdi         #, _38
 # C:/Program Files/w64devkit/lib/gcc/x86_64-w64-mingw32
    /15.2.0/include/c++/bits/stl_vector.h:106:      :
    _M_start(), _M_finish(), _M_end_of_storage()
        vmovdqu XMMWORD PTR [rbx], xmm6  # MEM <vector(2)
           long long unsigned int> [(int * *)__first_49],
           tmp167
```

Полный файл ассемблерного кода лежит в репозитории и доступен по ссылке.

7 Вывод по результатам выполнения задания

Результаты показывают значительное улучшение производительности:

- Применение ключей компилятора позволило ускорить программу в 2.2 раза за счет автоматических оптимизаций.
- Полуавтоматическая оптимизация дала существенный прирост производительности - программа ускорилась в 6 раз по сравнению с исходной версией.

Анализ ассемблерного кода показал, что OpenMP эффективно распараллелил вычисления между ядрами процессора. При этом компилятор применил оптимизации для работы с памятью и векторные инструкции для инициализации структур данных.

Значительное ускорение (в 6 раз) демонстрирует, что для задач с интенсивным обращением к памяти, таких как транспонирование матриц, применение OpenMP является эффективным методом оптимизации.